Chapter 10

Object Oriented Programming

ECE 264 – Object-Oriented Software Development

Core Objectives for this chapter

- Learn the difference between types of programming
- Define OOP
- Define the pillars of OOP
- Use OOP
- Define your own class and objects

The different types of programming

- Object oriented programming
- Procedural
- functional

- Paradigm organizes code around the idea of objects and classes
- Focuses on programming in sequences or steps
- Treats computation as the evaluation of mathematical functions

Procedural example

```
struct Circle {
    double radius;
};

double calculateArea(struct Circle c) {
    return 3.14159 * c.radius * c.radius;
}

// Usage
struct Circle myCircle = {5.0};
double area = calculateArea(myCircle);
```

Functional

```
(* Define a custom data type for Shapes *)
datatype shape =
    Circle of real
  | Square of real;

(* Function using Pattern Matching *)
fun calculateArea (Circle r) = 3.14159 * r * r
  | calculateArea (Square s) = s * s;

(* Usage *)
val myCircle = Circle 5.0;
val area = calculateArea myCircle;
```

OOP

```python
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def calculate_area(self):
        return 3.14159 * self.radius ** 2

# Usage
my_circle = Circle(5.0)
print(my_circle.calculate_area())
```

Object Oriented Programming Defintion

- Allows programmers to program in terms of real-world objects. The ability to use templates.
- Has four basic pillars:
  - Encapsulation – provides security through hiding internal workings
  - Abstraction- shows only the essential features of an object, hides the rest
  - Inheritance – allows classes to adopt features from other classes
  - Polymorphism – allows different objects to respond to the same command in a unique way

abstraction

- Hides complexity: focuses on what an object does rather than how
- Interface vs Implementation: provides a simple interface while hiding complex logic underneath
- Allows users to interact with the inputs and outputs without needing to understand the internal mechanics

Abstract class in python

```python
from abc import ABC, abstractmethod

# The Abstract Class (The Blueprint)
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

# The Concrete Class (The Implementation)
class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side * self.side

    def perimeter(self):
        return 4 * self.side

# This will fail! (You cannot instantiate an abstract class)
# my_shape = Shape()

# This works!
my_square = Square(5)
print(my_square.area())
```

Java Abstract Class Example

```java
// abstract class (cannot be instantiated)
abstract class Shape {
    String color;

    // A standard, concrete method with implementation
    void showColor() {
        System.out.println("The color is " + color);
    }

    // Abstract method (no implementation)
    // Subclasses MUST override this method.
    abstract double calculateArea();
}

// Concrete subclass (implementing the abstract method)
class Circle extends Shape {
    double radius;

    public Circle(String color, double radius) {
        this.color = color;
        this.radius = radius;
    }

    // Overriding the abstract method
    @Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }
}
```

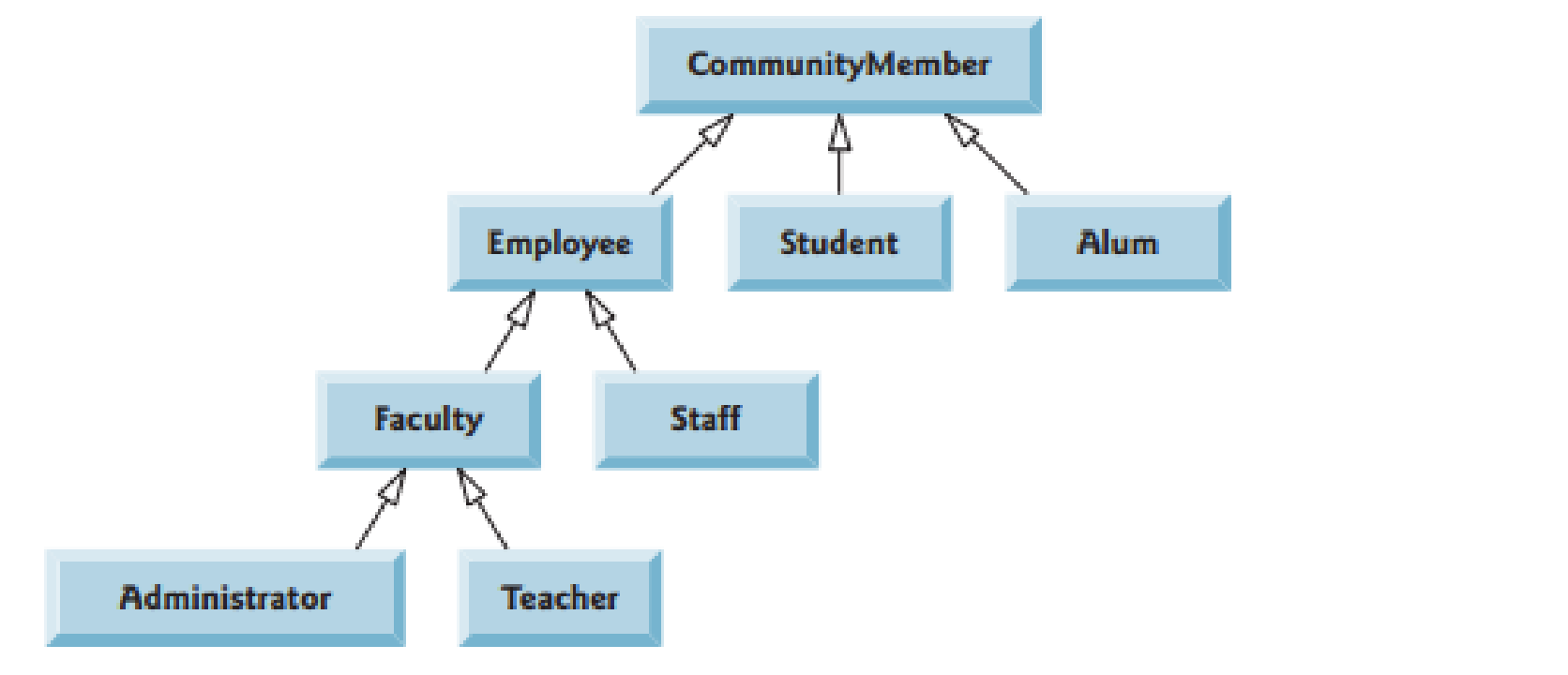
Inheritance

- Allows classes to adopt properties from parent classes
- Is an "IS-A" relationship, example Circle IS-A shape, a Car IS-A vehicle
- allows for extremely reusable code

Inheritance (cont'd)

Base class	Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
BankAccount	CheckingAccount, SavingsAccount

Inheritance Graph



Python inheritance example

```python
# Superclass
class Employee:
    def __init__(self, name):
        self.name = name

    def clock_in(self):
        print(f"{self.name} has clocked in.")

# Subclass (inherits from Employee)
class HourlyEmployee(Employee):
    def __init__(self, name, hourly_rate):
        super().__init__(name) # Initialize the parent part
        self.hourly_rate = hourly_rate

    def calculate_weekly_pay(self, hours):
        return hours * self.hourly_rate

# Usage
contractor = HourlyEmployee("Bob", 50.0)
contractor.clock_in() # This method exists in the Parent!
```

Encapsulation

- Bundles data and methods: combines attributes and similar functions into a single unit (the class)
- Data hiding: restricts direct access to an objects certain state
- controls access to view or modify data through getter and setter functions

Encapsulation example in python

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        # Private attribute: cannot be accessed directly as .__balance
        self.__balance = balance

    # Getter method: Controlled way to see the data
    def get_balance(self):
        return f"Current balance: ${self.__balance}"

    # Setter method: Controlled way to change the data (with validation!)
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited ${amount}")
        else:
            print("Invalid deposit amount!")

# Usage
account = BankAccount("Alice", 1000)

# This will raise an AttributeError!
# print(account.__balance)

# Correct way to interact:
account.deposit(500)
print(account.get_balance())
```

Protected vs private

_variable

- Subclasses and intended to use these
- Child classes can see it exactly as it is in the parent class
- Can be used in helper methods in a child class to extend functionality from parent

__variable

- Is isolated
- Not inherited in a useful way
- This is used to make sure a subclass cannot overwrite a critical value of the parent class

```python
class Parent:
    def __init__(self):
        # Protected: "A hint for the family"
        self._protected_item = "Family Heirloom"

        # Private: "Strictly for this class"
        self.__private_item = "Private Diary"

    def show_private(self):
        # The parent CAN access its own private data
        print(f"Parent accessing: {self.__private_item}")

class Child(Parent):
    def access_parent_data(self):
        # 1. Accessing Protected: SUCCESS
        # Child sees '_protected_item' exactly as it is named.
        print(f"Child accessing protected: {self._protected_item}")

        try:
            # 2. Accessing Private: FAIL
            # Python looks for '_Child__private_item' which does not exist.
            print(f"Child accessing private: {self.__private_item}")
        except AttributeError:
            print("Child CANNOT access private_item (AttributeError)")

# --- Execution ---
p = Parent()
c = Child()

c.access_parent_data()

# 3. Accessing from OUTSIDE:
print(f"Outside accessing protected: {p._protected_item}") # Works (but bad practice)

try:
    print(p.__private_item) # Fails immediately
except AttributeError:
    print("Outside CANNOT access private_item")

# 4. The "Backdoor" (Name Mangling)
# You can still get it if you know the secret internal name:
print(f"Backdoor access: {p._Parent__private_item}")
```

Polymorphism

- Meaning many forms
- Many classes can have methods with the same names but different logic
- Dynamic Typing: if an object "walks like a duck and quacks like a duck, then it is a duck"
- Method overriding: when a subclass takes a method from parent class and redefines

Polymorphism Python Example

```python
class Animal:
    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

# Polymorphism in action:
# A single function that works for ANY animal
def make_it_talk(animal):
    print(animal.speak())

animals = [Dog(), Cat()]

for a in animals:
    make_it_talk(a) # Same function call, different results!
```

Designing a class

- Think of a simple name
- Think about if you need to make a super class to design other objects
- Think of simple attributes that you can pass along to different subclasses
- Think of simple methods to pass to different classes if possible

attributes

- Attributes are the data fields you want to give to an object
- These are the traits they are
- For example: people have height, weight, hair color, eye color, etc.

Getters and setters

- These refer to functions that return info or change attributes for an object
- Getters fetch the information from the object
- Setters change information about the object
- Getters and setters are used because you should not just change the objects attributes directly, it is bad practice

Other methods

- Methods refer to the functions that are bound to classes and objects
- Methods usually perform some action on objects
- In our person example
  - Exercise -> increases heart rate and reduces weight
  - Breathing -> increases oxygen and decreases heart rate

Case Study 1: the human class

- What should we start with first?
- Should there be something before human?
- What should we include as attributes?
- What should we inlcude as methods?

Case study 2: The athlete class

- How do we start with the athlete class?
- How can we make different sports?
- How to design the class in a way other people can use it differently?

Case Study 3: The Monster Class

- We are making a game and we want to make some monsters to fight
- What class should we start with?
- What should we include?
- What methods should we add?